LLM-Enhanced Intrusion Detection for Containerized Applications: A Two-Tier Strategy for SDN and Kubernetes Environments

Sarantis Kalafatidis(✉), Nikos Papageorgopoulos, Andreas Kartakoullis, and Giannis Ledakis

Ubitech Ltd., Computing Systems and Software Engineering (CSE) Group, Athens, Greece
{skalafatidis,npapageorgopoulos,akartakoullis,gledakis}@ubitech.eu

Abstract. Traditional Intrusion Detection Systems mainly rely on rule-based mechanisms, which are limited in detecting unknown attack patterns and often result in false positives or false negatives. Deep packet inspection, although effective, demands significant computational resources as it requires processing large network traffic data volumes. Similarly, AI-based solutions frequently consume excessive resources, making them impractical for production environments, especially those with resource constraints or high-volume traffic patterns. In this paper, we propose and investigate a two-tier intrusion detection strategy, targeting an optimal balance between effective threat detection and resource efficiency. Our approach combines lightweight statistical monitoring for continuous anomaly detection with on-demand LLM-based traffic analysis, activating deep inspection only when necessary. We implement and evaluate two systems that enable centralized data collection among distributed containers, one for SDN-based environments utilizing the OpenFlow protocol and another for Kubernetes-based infrastructures utilizing Cilium-Hubble integration. Both systems initiate deep traffic analysis via LLMs only when statistical anomalies are detected, targeting low overhead while maintaining high detection accuracy. We demonstrate the efficiency of our approach through real-world attack scenarios, showing performance in detecting network-based attacks such as DDoS, port scans, and brute-force attempts.

Keywords: Intrusion Detection · Large Language Models · Cybersecurity · Kubernetes · SDN · Anomaly Detection

1 Introduction

Cloud computing has transformed digital service deployment, offering flexibility, cost-efficiency, and scalability, enabling organizations to dynamically provision resources on demand, allowing systems to adapt instantly to dynamic workload

© The Author(s) 2025
F. Skopik et al. (Eds.): ARES 2025 Workshops, LNCS 15998, pp. 55–73, 2025.
https://doi.org/10.1007/978-3-032-00642-4_4

changes. Microservices architecture has significantly evolved this trend, enabling applications to be broken down into lightweight, independent components (e.g., containers) that can be efficiently deployed, scaled, and managed through orchestration platforms like Kubernetes. However, such cloud service environments frequently become targets for attackers who aim to cause service disruptions or Quality-of-Service (QoS) degradation [1], potentially impacting multiple interconnected components and disrupting business operations across the distributed infrastructure [2].

Service providers face significant security challenges arising from inherent vulnerabilities in the services themselves (e.g., API operations exploitable for resource exhaustion, or misconfigured rate limits), as well as frequent network attacks that may be hidden within high volumes of network traffic [3]. Additionally, centralized security monitoring is necessary across distributed services, where the complexity of management increases the difficulty of detecting and mitigating attacks. For example, in a microservices architecture where dozens of containerized applications communicate across multiple cloud environments, a distributed denial-of-service (DDoS) attack might target several entry points simultaneously [4]. Moreover, attack detection and mitigation near real-time are important in order to reduce the attacks' impact on the services as much as possible.

Network-based attacks on cloud-provided services [5], such as DoS attacks across different network protocols (e.g., TCP, UDP, HTTP/1, and HTTP/2 [6]), alongside port scanning and SSH brute-force attempts, present multiple technical obstacles in security management: For instance: (i) large-scale deployments generate high volumes of network traffic, which must be processed with minimal computational overhead; (ii) malicious traffic often mimics legitimate service communication, making differentiation difficult; (iii) detection systems must continue working effectively during container scaling events; (iv) signals of a distributed attack may be spread across multiple containers or nodes, requiring mechanisms to correlate events across the system; and (v) minimizing false positives while maintaining a high detection rate is essential to avoid unnecessary alerts and maintain trust in the system.

Targeting these challenges, traditional Intrusion Detection Systems (IDS) [7] primarily rely on rule-based mechanisms for threat detection and classification. For example, a conventional IDS might use a predefined signature to identify a TCP SYN flood attack by monitoring for an unusually high number of incomplete TCP handshakes from multiple source IPs within a short timeframe. However, these systems present significant limitations when confronting unknown attack patterns. These conventional approaches fail to identify novel threats that don't match predefined signatures, frequently resulting in false positives or false negatives. As threat landscapes evolve, the static nature of rule-based detection becomes increasingly insufficient to protect modern distributed service architectures against sophisticated and emerging attack vectors. For example, in newer protocols like HTTP/2 [6], attacks can exploit multiplexing capabilities (i.e., multiple requests and responses simultaneously over a single TCP connec-

tion) and header compression vulnerabilities (e.g., HPACK Compression attack) to bypass traditional rule-based systems.

AI-based intrusion detection offers several advantages in securing containerized systems. Such methods can adapt to constantly changing environments and traffic patterns without requiring manual rule updates. They are also capable of detecting complex, multi-stage attacks by recognizing behavioral patterns. Additionally, they improve over time by learning from new data and are better suited to correlating distributed attack signals across large-scale container networks [8].

However, several technical challenges limit the effectiveness of AI-based detection. Complex AI models can impose high computational overhead [8], which impacts container performance. Moreover, AI systems require substantial training data that accurately reflects the target environment [9], something difficult to maintain in fast-changing container deployments. As workloads evolve, models can experience drift, reducing accuracy over time. Real-time threat detection also imposes strict performance requirements on AI models. Balancing detection accuracy with resource consumption is a core design trade-off in these systems.

Large Language Models (LLMs) have emerged as a promising tool for real-time network attack detection. A recent survey [10] explores their role in cybersecurity. LLMs can understand the context of network events, allowing them to identify abnormal patterns without the need for extensive labeled training data. They can also interpret and correlate information from different sources (e.g., logs, telemetry, metadata) and provide human-readable explanations for potential threats. Additionally, their zero-shot learning capabilities help detect previously unseen attacks. Their ability to generate natural-language summaries of threats is especially helpful for incident response.

Nonetheless, deploying LLMs in cloud environments for near-real-time attack detection and mitigation presents significant limitations. They require significant computational resources and often introduce latency that may hinder timely detection. Network traffic data requires preprocessing and transformation into suitable input formats for LLMs, adding complexity and processing overhead to the detection pipeline. Furthermore, LLM model selection involves tradeoffs between detection accuracy and computational efficiency. Consequently, ensuring LLM responses are rapid and reliable enough for near real-time attack mitigation remains an open challenge.

This paper proposes a two-tier intrusion detection strategy for containerized applications, combining lightweight statistical monitoring with on-demand LLM-based traffic analysis. The main aspect is the balanced approach that provides continuous monitoring through simple statistical models (first tier) while only activating packet LLM analysis (second tier) when anomalies are detected.

To this end, we propose and evaluate two LLM-enhanced IDS targeting for near real-time attack detection in container-based environments: (i) an SDN-based approach utilizing the OpenFlow protocol and (ii) a Kubernetes-based approach utilizing Cilium-Hubble [11] integration. We measure their performance in terms of detection accuracy and response speed. Furthermore, we analyze

each approach's performance benefits and complexity trade-offs. Our key contributions include: (1) a centralized approach to network traffic monitoring with on-demand AI analysis, (2) efficient resource usage through strategic PCAP traffic sampling, and (3) analysis of two architectural implementations on different container orchestration environments.

The systems under investigation share the following common features: Both systems use a centralized approach for data collection. The LLM-enhanced detection mechanism is located at the central control point. Additionally, both systems employ simple statistical models to detect spikes in network statistics and on-demand activate preprocessing mechanisms to extract necessary data from PCAP files for analysis by LLM analysis systems. To reduce resource overconsumption, our strategies implement sampling of PCAP files; when a spike is detected, the system captures samples from the traffic and stores them in PCAP files for further analysis.

The remainder of this paper is organized as follows: Sect. 2 presents our proposed systems in detail, explaining the architecture and operational flow of both the SDN-based and Kubernetes-based solutions. Section 3 provides implementation details, including the technologies, frameworks, and configuration parameters used in our experimental setup. Section 4 presents our performance evaluation methodology and results, comparing the effectiveness of our approaches across multiple attack scenarios. Finally, Sect. 5 concludes the paper with a summary of our contributions and discusses potential directions for future work.

2 Proposed Systems

Our research investigates two approaches for LLM-based intrusion detection, designed specifically for containerized environments. Both approaches share fundamental architectural principles while implementing different technologies and methodologies appropriate to their respective deployment environments. The systems are designed to address the challenges of detecting network-based attacks (especially DoS attacks and patterns) in containerized infrastructures while minimizing resource utilization. Both proposed systems share the following design principles:

- Two-Tier Detection Strategy: Both systems implement a novel two-tier detection approach, i.e., lightweight statistical monitoring for continuous observation, followed by AI analysis only when needed, targeting to create an optimal balance between comprehensive security and resource efficiency.
- On-demand AI Intervention: Unlike traditional security solutions that often introduce significant overhead, our approaches specifically address resource constraints in containerized environments through selective data processing and on-demand AI activation, making them practical for production deployments with heavy load traffic.
- Centralized Monitoring with Distributed Awareness: Both systems collect network traffic data from a centralized point while maintaining visibility into

distributed container interactions, enabling comprehensive attack detection among network elements (e.g., containers and network nodes).
- Statistical Anomaly Triggers: To conserve resources, both systems employ lightweight statistical models to identify potential anomalies before activating more resource-intensive AI analysis components.
- Strategic Traffic Sampling: When anomalies are detected, both systems implement traffic sampling mechanisms to capture relevant pcap data without overwhelming storage or processing capabilities.

A detailed description of each subsystem follows.

2.1 System A: SDN-Based Solution

System A is an AI-enabled IDS that utilizes Software-Defined Networking (SDN) capabilities to protect containerized applications across different servers. This SDN implementation is grounded on [12], and [13]. The system's primary goal is to detect network-based attacks by combining three key features: (i) centralized collection of network statistics through the SDN network; (ii) container resource monitoring focused on CPU and memory consumption, and (iii) a two-tier detection approach that uses statistical analysis for anomaly detection followed by on-demand LLM-based analysis.

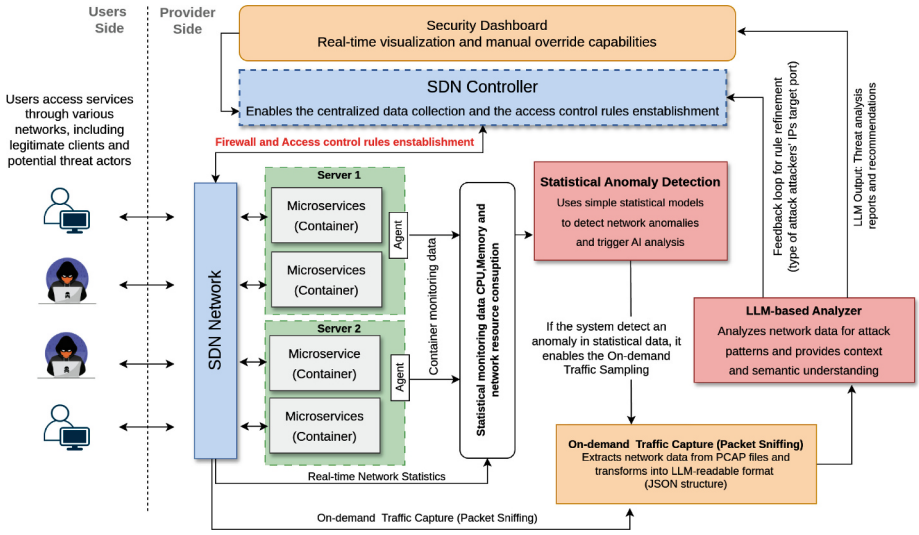

Fig. 1. SDN-based AI-IDS workflow.

Figure 1 illustrates the overview of System A. At the core of the system, on the provider side, are containerized service applications hosted on different servers. On the user side, there are clients requesting services, among whom

malicious actors may exist. To detect malicious activities primarily related to network attacks, the system follows this pipeline:

A. Data Collection Subsystem

Regarding network statistics collection, the system's central component is the SDN Controller (in this implementation, we utilize the Floodlight controller [14]), which enables centralized data gathering from all SDN network devices. The system uses virtual SDN switches based on OpenVSwitch software to which each container is connected. The architecture employs virtual SDN switches based on OpenVSwitch software, with each container directly connected to these switches. For SDN network development and configuration, we use the Containernet framework [15] (an extension of Mininet), which provides both Python Docker API integration and SDN virtual switch deployment capabilities. Network statistics are collected via the OpenFlow protocol, capturing key metrics including active flows in the network, flow duration, protocol distribution, and bandwidth utilization (bytes transferred).

Regarding Docker containers monitoring statistics, custom agents are utilized. On each server, an agent collects statistical data related to CPU and memory consumption of each hosted container. This tool is based on the Dockerstats application and forwards the data to the central system where the SDN controller is located.

All network and computing resource statistics are stored in time-series databases for subsequent anomaly detection analysis.

B. Statistical Anomaly Detection

This component supports multiple mechanisms for attack detection, including:

- Threshold-based detection for identifying time instances (e.g., CPU or number of flows peaks) in real streaming data
- Clustering and change-point detection to identify changes in incoming flow characteristics

The current implementation adopts the approach presented in [13], utilizing simple statistical methods, specifically Exponential Moving Average (EMA) and Auto-Regressive Integrated Moving Average (ARIMA), since, according to our initial investigation, it seems the most suitable for such kinds of attacks (i.e., DoS) due to spikes in resource utilization statistics. More precisely, the SDN controller collects the number of active flows periodically (every 0.5 s) and creates time-series data. The anomaly detection system uses the EMA method, which focuses on recent data by assigning weights to new incoming data and checks if new records deviate significantly from expected values. The expected value range is calculated using the formula: EMA + (standard deviation $\times$ threshold). In the current implementation, the threshold is set to 1.5, and the EMA's alpha-parameter, which determines the importance of the last record, is 0.5.

C. On-demand Traffic Sampling

When the system detects an anomaly in the data related to a container, it initiates packet capture on the server where the container is located using

TCPdump tool. The total duration of the PCAP capture is limited to 1 s, and each PCAP file is limited to 1,000 packets to enable rapid and effective analysis by the LLM mechanism. After collection, the PCAP files are processed using tshark [16] to convert them into JSON format, which is then pushed to LLM for advanced analysis and context-aware threat detection.

D. LLM-based Analysis

The system employs an LLM-based analysis approach that serves as the advanced tier of the two-tier detection strategy. This component is triggered automatically when the Statistical Anomaly Detection activates the On-demand Traffic Sampling. The analysis workflow operates as follows:

First, the system maintains a repository of JSON representations from normal traffic patterns (nominal traffic) to establish a baseline of legitimate communication behaviors for each container. Nominal traffic refers to the typical network communication patterns observed during normal operations when no attacks or anomalies are present. These patterns include regular communication flows, expected protocols, typical packet sizes, and standard connection durations that represent the container's legitimate behavior under normal conditions. When an anomaly is detected, the newly captured PCAP files are converted to JSON format (through the on-demand traffic sampling tool) using tshark and forwarded to the LLM-based analyzer. The LLM receives a structured prompt containing:

1. The JSON representation of the current traffic event under investigation
2. Historical communication patterns observed during nominal operations
3. Instructions to assess the probability of a security threat

The prompt template is designed for contextual threat analysis:

LLM Prompt Template

Analyze the following pcap file: {json.dumps(event_json)}.
Consider that this container typically communicates with: {", ".join(nominal_json)}.
Based on these patterns and the current event context, calculate the probability this represents an actual security threat versus normal operational behavior.
Return your response as JSON: {"threat_probability": <float>, "justification": <string>}

This approach utilizes the LLM's pattern recognition and contextual understanding capabilities to:

1. Compare current traffic patterns against established baselines
2. Historical communication patterns observed during nominal operations
3. Provide explainable reasoning for its threat assessment
4. Deliver a quantifiable threat probability that can be used for automated decision-making

For our implementation, we focus on utilizing models from the Together AI platform [17], particularly LLAMA and Mistral models, while also evaluating GPT-4 for comparative analysis. These models offer an effective balance between performance, cost, and inference speed for real-time security applications. The structured JSON response format enables seamless integration with the broader security infrastructure, allowing automated responses based on threat probability thresholds. The justification field provides valuable context for security analysts to understand the reasoning behind the threat assessment, enhancing the system's explainability and trustworthiness.

By combining statistical anomaly detection with LLM-based deep analysis, the system achieves an optimal balance between continuous lightweight monitoring and resource-intensive deep inspection, only when necessary. This approach is particularly effective for detecting attacks that show spikes in resource utilization, such as DoS attacks. The LLM helps us understand more detailed information about the attack, including attack type and the attacker's IP address, enabling the implementation of mitigation actions such as blocking the attacker's IP. Future work will focus on deploying on-premises LLMs to eliminate external dependencies, which would further enhance security by keeping sensitive network traffic data within the organization's infrastructure while potentially reducing latency in threat detection and response.

E. SDN-enabled Mitigation Actions

The LLM-based system demonstrates significant capability in identifying attacker IPs amidst nominal traffic. For example, an output from GPT-4 following a TCP SYN flooding attack contains this analysis:

LLM Output

From line 27 onwards, we observe that:
The address 192.168.122.1 is sending multiple TCP SYN packets to 192.168.122.208 on port 30080. Similarly, 10.244.1.244 is doing the same to 10.244.1.74 on port 80.
Key observations:
Hundreds of SYN packets within fractions of a second from different source ports. No normal TCP sequence (SYN → SYN/ACK → ACK). Instead, there is a massive transmission of SYN packets from different source ports, characteristic of a SYN flood attack. SYN, ACK is returned from the servers, but there appears to be no completion of the handshake, i.e., no ACK response.
Conclusion: This is a DoS attack of the SYN Flood type, or alternatively port scanning with massive SYN packets (e.g., from a tool like nmap -sS).
Proposed Mitigation Action: Block/Rate-limit IP 192.168.122.1 via firewall

Our system utilizes the rapid implementation of security policies in the network through SDN capabilities. For instance, the Floodlight controller includes Security Modules with sets of conditions that allow or deny traffic flows, which can be rapidly deployed at any network point (SDN-node). These modules facilitate immediate response to detected threats through programmable network policies. At the current stage of our research, policy implementation is per-

formed manually through the controller's API. Once we thoroughly evaluate the accuracy of LLMs in detecting attacker IPs, we will proceed with developing an automated pipeline for immediate mitigation following detection.

2.2 System B: Kubernetes-Based Solution

In this section, we describe System B, our Kubernetes-based solution that adopts the same two-tier detection approach as System A, but is specifically adapted for Kubernetes environments. Like System A, System B implements a lightweight statistical monitoring layer for continuous observation, followed by on-demand AI analysis only when anomalies are detected. However, System B takes advantage of the inherent benefits of Kubernetes for container orchestration to create an adaptable security solution for containerized microservices. Our approach focuses on integrating detection and mitigation strategies within the native Kubernetes orchestration environment, providing security features without requiring additional configuration from users deploying their services. After system deployment, users can seamlessly deploy their containerized services within the cluster while benefiting from built-in threat protection capabilities.

A. Architecture Overview

As illustrated in Fig. 2, System B implements a security architecture that consists of several integrated components designed specifically for Kubernetes environments. The key components include:

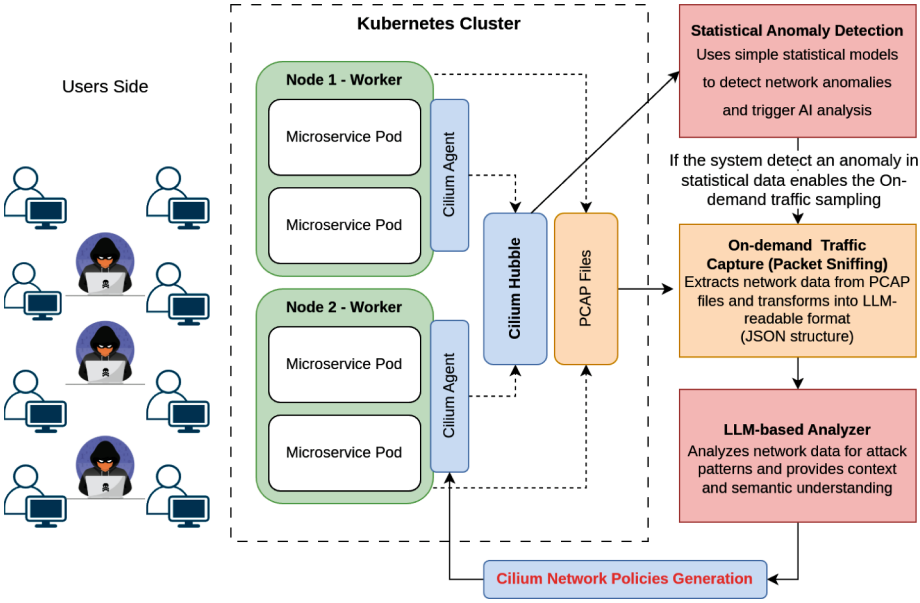

Fig. 2. Kubernetes-based LLM-IDS workflow.

1. Cilium-Hubble Network Monitoring: Acting as the central data collection mechanism, Cilium with Hubble provides comprehensive visibility into network communications between pods across the entire Kubernetes cluster. Cilium, operating as a Container Network Interface (CNI) plugin, facilitates both the collection of network telemetry and the enforcement of security policies.
2. Statistical Anomaly Detection: This lightweight component continuously analyzes network flow metrics collected by Hubble to identify potential anomalies in traffic patterns. Similar to System A, it uses Exponential Moving Average (EMA) to detect significant deviations in traffic volume, connection rates, or protocol distributions that might indicate attack behaviors.
3. On-demand Traffic Capture: When statistical anomalies are detected, this component triggers targeted packet capture operations to collect samples of the suspicious traffic for deeper analysis. The system implements strategic sampling to minimize resource consumption while capturing sufficient data for accurate threat assessment.
4. LLM-based Analyzer: The captured network data is processed into LLM-readable JSON format and submitted to the LLM component for contextual analysis, similar to System A. However, this implementation utilizes Kubernetes-based context information, such as pod-to-pod intercommunication data and service relationships.
5. Cilium Network Policy Generator: Based on the LLM's threat assessment, this component enables the automatic generation and establishment of Kubernetes Network Policies and Cilium-specific security rules to mitigate identified threats.

A detailed description of the subsystems follows.

B. Data Collection and Monitoring

The data collection mechanism in system B depends on the Cilium-Hubble integration for centralized monitoring. Specifically, Cilium is deployed as a DaemonSet across all cluster nodes, with Hubble providing the observability layer. This configuration enables detailed flow monitoring between pods, capturing key metrics including: (i) source and destination pod identifiers and IP addresses; (ii) protocol information and port numbers; (iii) flow duration and data volume; and connection states and protocol-specific attributes. Regarding the CPU and Memory resource consumption data from pods, the Kubernetes Metrics API is utilized, eliminating the need for additional monitoring agents within containers. All collected metrics are stored in a time-series database (Prometheus) deployed within the cluster, enabling efficient querying and analysis for both real-time detection and historical trend analysis.

C. Statistical Anomaly Detection and On-demand Traffic Capture

The anomaly detection component in System B employs similar statistical methods to System A, but is integrated for Kubernetes environments. For example, based on Cilium-Hubble collected data the EMA method is applied for each data type e.g., HTTP traffic, TCP traffic, etc. When statistical anomalies are detected, the system triggers targeted packet capture operations on the specific node (e.g., Kubernetes worker) where the malicious traffic is forwarded.

The total duration of the PCAP capture is limited to 1 s, and each PCAP file is limited to 1,000 packets to enable rapid and effective analysis by the LLM mechanism. After collection, the PCAP files are processed using tshark [16] to convert them into JSON format, which is then pushed to LLM for advanced analysis and context-aware threat detection.

D. LLM-based Analysis

The LLM-based analysis component in System B follows the same approach as in System A, with the key difference being that the nominal traffic pcaps include: (i) the pod-to-pod intercommunication patterns and (ii) traffic forwarded through Kubernetes exposed ports. This traffic context allows the LLM to understand both internal cluster communication and external access patterns, providing a more complete baseline for anomaly assessment.

Similar to System A, the system maintains a repository of JSON representations from normal traffic patterns to establish a baseline of legitimate communication behaviors. When an anomaly is detected, the newly captured PCAP files are converted to JSON format and forwarded to the LLM-based analyzer with contextual information about the Kubernetes environment.

The LLM receives a structured prompt containing the current traffic event under investigation, historical communication patterns, and instructions to assess the probability of a security threat. The system then processes the LLM's response to determine appropriate mitigation actions within the Kubernetes environment.

E. Kubernetes-native Mitigation

Based on the LLM's threat assessment, the system generates Kubernetes Network Policies that restrict pod communications to prevent malicious traffic. For example:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: block-attack-source
  namespace: target-namespace
spec:
  podSelector:
    matchLabels:
      app: affected-service
  ingress:
  - from:
    - ipBlock:
        cidr: 0.0.0.0/0
        except:
        - 192.168.122.1/32  # Block detected attacker IP
```

Also, the system utilizes Cilium's security features, such as Layer 7 protocol awareness (HTTP, gRPC, Kafka).

3 Experimentation Methodology

Our experimentation methodology is based on real infrastructure, considering real attack-generated data. We evaluated the effectiveness of the proposed systems through realistic experimental deployments in order to demonstrate their ability to provide online threat detection in both SDN-based and Kubernetes infrastructures.

3.1 Experimental Setup

This subsection describes the common approaches in the evaluation of both systems. We utilized two containerized applications as targets for our experiments:

1. Metasploitable2: Selected as a target service because it provides multiple vulnerable services such as FTP, SSH, web applications with SQL injection vulnerabilities, and database instances with known security flaws
2. NGINX Docker Server: Based on the Docker container dockette/nginx, specifically designed to offer HTTP/2 services.

We implemented various attack scenarios to evaluate the detection capabilities of our systems, with particular focus on attacks that generate network and CPU spikes, making them potentially detectable by statistical-based anomaly detection mechanisms.

Table 1. Attack Scenarios Used in Experimental Evaluation

Attack Type	Description	Target
UDP Flood	Overwhelming target with UDP packets, causing resource exhaustion	Metasploitable 2
Nmap Port Scan	Port scanning for target service	Metasploitable 2, NGINX
TCP Flood	Massive TCP connection attempts	NGINX, Metasploitable 2
HTTPv1 Flood	Overwhelming web servers with HTTP sessions	Metasploitable 2
ICMP Flood	Bandwidth over-consumption with ICMP echo	Metasploitable 2
HTTPv1 Slowloris	Slow-rate HTTP attack keeping connections open	Metasploitable 2
SSH Bruteforce	Authentication attack attempting multiple passwords	Metasploitable 2
HTTP/2 Ping Flood	Exploitation of HTTP/2 PING frames	NGINX
HTTP/2 Slow Get Flood	Slow HTTP/2 requests flooding	NGINX

For both systems, we utilized the deployment methodology described in the previous section for each environment (SDN-based and Kubernetes-based). To

evaluate detection capabilities under various conditions, we simulated nominal traffic alongside attack traffic. The nominal traffic consisted of legitimate HTTP client requests and SSH traffic with three different traffic patterns, similar to the work [12]: (i) Gradually increasing: Linear growth in request volume to simulate growing user engagement; (ii) Gradually decreasing: Linear reduction in request volume to simulate declining activity periods; and (iii) Stable: Constant request volume to establish a baseline detection threshold.

All experiments were conducted with identical hardware specifications to ensure fair comparison between the SDN-based and Kubernetes-based approaches. For each attack scenario, we measured detection accuracy, time-to-detection, and system resource utilization.

This section presents the evaluation of System B, the Kubernetes-based detection architecture described in Sect. 2.2. The focus is on demonstrating the full detection pipeline: from anomaly detection and packet capture to LLM-based analysis and mitigation.

3.2 LLM-Based Traffic Analysis

This subsection describes the LLM-based traffic Analysis strategy, which was utilized in both systems. Regarding the LLM-input, both systems utilize the On-demand Traffic Sampling which was described in Sect. 3.1.C. Specifically, captured packets are parsed into structured JSON format using the PyShark library, a Python wrapper over TShark. Each packet is converted into a JSON object that includes the essential fields needed for traffic pattern analysis, such as timestamp, IP addresses, ports, protocols, packet size, and protocol-specific flags. This preprocessing ensures that only relevant features are passed to the LLM, reducing input complexity and improving reasoning focus. An example of a parsed packet is shown below:

```
{
  "index": 3,
  "timestamp": 1.112393,
  "src": "192.168.122.1",
  "dst": "192.168.122.208",
  "protocol": "TCP",
  "length": 74,
  "info": "54304 → 22 [ACK] Seq=1 Ack=77 Win=1635 Len=0"
}
```

The analysis component uses the Mixtral-8×7B model, accessed via the Together API. This model was selected for its reasoning capabilities, including cause-effect inference, pattern comparison, and protocol-aware interpretation of traffic behaviors. Although not deployed locally, the API provides access

to high-capacity inference infrastructure capable of handling complex prompts, albeit with higher latency due to network and service delays.

The LLM receives a structured prompt containing: (i) a summary of suspicious traffic (JSON from the recent pcap capture); (ii) a summary of nominal container communication patterns (historical baseline); and (iii) instructions to assess whether the suspicious pattern represents malicious activity. A typical prompt format is:

LLM Input

```
Below are summaries of two PCAP files:

Suspicious traffic:
{json.dumps(suspicious_summary, indent=2)}

Normal container behavior:
{json.dumps(nominal_summary, indent=2)}

Analyze whether the suspicious traffic is malicious.
Compare connection patterns, protocols, ports, and IP activity.

Return ONLY a JSON object in this format:
{
  "threat_detected": <true/false>,
  "attack_type": "<short string like 'SYN Flood' or 'Normal'>",
  "justification": "<one-sentence explanation>",
  "mitigation": "<concrete action such as 'Block IP'>"
}
```

This design allows the model to incorporate justification within its reasoning capabilities beyond simple statistical flags. For example, instead of merely noting repeated connections: *"Multiple connections observed on destination port 8472."* the LLM responds with deeper insight: *"The suspicious traffic contains multiple connections from different source ports to the same destination port (8472) on the target IP (192.168.122.208). This pattern is indicative of a port scan aimed at probing known service ports to discover accessible endpoints, which deviates from typical container communication behavior."*

The model's output contains fields for threat presence, attack classification, explanation, and mitigation. For example, for a detected port scan:

LLM Output

```
{
   "threat_detected": true,
   "attack_type": "Port Scan",
   "justification": "The suspicious traffic contains multiple
 connections from different source ports to the same destination
 port (8472) on the target IP (192.168.122.208). This pattern is
 indicative of a port scan, where an attacker is attempting to
 identify open ports and services on the target system.",
   "mitigation": "Block the source IP (192.168.122.28) at the
 firewall level, and alert the security operations center (SOC)
 of the potential port scan. Consider implementing rate limiting on
 incoming connections to the target IP to prevent further scanning
 attempts."
}
```

Based on the LLM's assessment as describes in 2.2, the system initiates a mitigation response. In this case, a Kubernetes NetworkPolicy is dynamically generated to block further communication from the identified source IP `192.168.122.28`. This targeted policy is applied through the Cilium agent, ensuring immediate enforcement at the network layer without manual intervention.

4 Evaluation Results

This section presents the performance evaluation of our two-tier LLM-enhanced intrusion detection systems in both SDN and Kubernetes environments. Our assessment focuses on two key dimensions:

4.1 Statistical Anomaly Detection Performance

The statistical anomaly detection component forms the first tier of our detection strategy, consisting the initial filter for identifying potential threats. We evaluated its performance across both proposed environments.

In the SDN-based environment (System A), the statistical anomaly detection demonstrated consistent and rapid response times across all attack types. Detection latency ranged between 1–1.5 s from the moment attacks were initiated (including the attack tool initialization time for tools like hping3 for DoS attacks or Hydra for SSH brute force attempts). This rapid detection capability can be attributed to the direct integration with the SDN controller's flow statistics collection mechanism, which provides near real-time visibility into network behavior changes.

However, the Kubernetes-based environment (System B) exhibited notably higher detection latency, with average detection times of approximately 6 s. We

observed instances where the system experienced extreme delays of up to 30 s before detecting anomalies. This significant difference can be attributed to the multi-component monitoring pipeline (Cilium - Hubble - Prometheus - Statistical API) that introduces additional processing overhead. The communication path between these components creates bottlenecks that delay the statistical analysis process. As part of future work, we plan to optimize this monitoring pipeline by implementing more direct integration with Hubble's observability capabilities to reduce this latency.

4.2 LLM Detection Performance

The second tier of our detection strategy involves semantic analysis of network traffic using an LLM. To evaluate this component, we collected PCAP samples for each attack scenario and analyzed them using the prompt and methodology described in Sect. 3.2. For this evaluation, we collected 10 PCAP files (each containing 1,000 packets) for each attack type and processed them through our LLM analysis.

Across all attack scenarios, the LLM consistently identified the presence of threats with the `"threat_detected": true`. Table 2, shows the LLM's response to the "attack_type" field, investigating whether the LLM successfully identifies the correct type of attack across multiple test samples. In the 1,000 packet captures, the LLM successfully recognized the attack type in the majority of cases. For instance, it correctly labeled attacks as "UDP Flood", "Nmap Port Scan", or "SYN Flood" depending on the scenario.

To reduce processing overhead, we evaluated LLM-detection accuracy using a dataset of 200 packet captures. The system successfully identified most attack types, though we observed minor detection failures primarily with HTTP/2 attacks. This shows that short PCAP samples may not capture enough evidence, and longer or more targeted captures can improve detection accuracy.

Table 2. LLM Attack Type Detection Results

Attack Type	LLM Response with 1000 packet input	LLM Response with 200 packet input
UDP Flood	UDP Flood	Data Flood
Nmap Port Scan	Nmap Port Scan	Nmap Port Scan
TCP Flood	SYN Flood	SYN Flood
HTTPv1 Flood	HTTP GET Flood	HTTP GET Flood
ICMP Flood	ICMP Ping Flood	ICMP Ping Flood
HTTPv1 Slowloris	HTTP GET Flood	HTTP GET Flood
SSH Bruteforce	SSH Bruteforce	SSH Bruteforce
HTTP/2 Ping Flood	HTTP/2 Ping Flood	HTTP GET Flood
HTTP/2 Slow Get Flood	HTTP GET Flood	HTTP GET Flood

All attacks were successfully executed from the virtual machine with external IP `192.168.122.1` targeting the services (Metasploitable2 and HTTP/2 NGINX) hosted on the virtual machine with IP `192.168.122.28`. During our initial testing, we observed that the LLM occasionally confused the attacker's IP address with the IP addresses of the services/victims. To address this issue, we refined our prompting strategy by explicitly providing the external IP addresses of the systems within the context. After implementing this enhancement, the system successfully identified the attacker's IP address across all attack types.

Regarding the LLM analysis pipeline detection time, we measured the execution of PCAP parsing and LLM processing. The malicious PCAP parsing required 0.33 s, and similarly, the nominal PCAP parsing took 0.33 s. However, the LLM processing itself consumed 4.53 s, resulting in a total execution time of 5.20 s for the complete analysis pipeline. It's important to note that the significant delay in LLM processing is primarily attributed to our use of the Together AI API for LLM inference. This external API dependency introduces network latency and queue waiting times that substantially impact the overall detection speed. The time measurements suggest that approximately 87% of the total execution time is spent waiting for and processing the LLM response. In future implementations, deploying an on-premises LLM solution would likely reduce this latency considerably, as it would eliminate network delays and external service dependencies.

5 Conclusion

This paper presented a novel two-tier approach for intrusion detection in containerized environments, combining lightweight statistical anomaly detection with on-demand LLM-based deep analysis for enhanced security monitoring. We proposed and experimented with an SDN-based and a Kubernetes-based solution targeting to effectively balance detection accuracy with resource efficiency. More precisely, the statistical first tier enables continuous monitoring with minimal overhead, while the LLM-based second tier provides detailed attack characterization and actionable mitigation recommendations.

The experimental evaluation of both SDN and Kubernetes implementations demonstrated the effectiveness of our approach in detecting various network-based attacks in a resource-efficient manner (i.e., our resource efficiency strategy was the on-demand PCAP capture mechanism), which only collected and analyzed network traffic when statistical anomalies were detected. This selective approach reduces the computational overhead compared to continuous deep packet inspection, as our systems monitored lightweight metrics (such as flow counts, CPU spikes, and connection patterns) and only triggered resource-intensive LLM analysis when these metrics indicated potential threats. Furthermore, according to the results, when these anomalies were detected, the LLM-based analysis provided detailed attack characterization and actionable mitigation recommendations.

Our future work will focus on: (i) enhancing the LLM-analysis component by fine-tuning models specifically for network security applications and expanding

the contextual information provided to improve attack classification accuracy; (ii) conducting extended performance comparisons between our approach and other IDS solutions, both AI-based and traditional; (iii) optimizing the statistical monitoring pipeline, particularly in the Kubernetes environment; and (iv) investigating and implementing an on-premises LLM solution to reduce the dependency on external APIs and minimize processing delays.

Acknowledgment. This paper has received funding by the Digital Europe Programme under grant agreement No. 101145861: "Uptake of Innovative Security-as-a-Service Solutions - CyberSuite".

References

1. Radain, D., Almalki, S., Alsaadi, H., Salama, S.: A review on defense mechanisms against distributed denial of service (DDoS) attacks on cloud computing. In: 2021 International Conference of Women in Data Science at Taif University (WiDSTaif), pp. 1–6. IEEE (2021)
2. Darwish, M., Ouda, A., Capretz, L.F.: Cloud-based DDoS attacks and defenses. In: International Conference on Information Society (i-Society 2013), pp. 67–71. IEEE (2013)
3. Praseed, A., Thilagam, P.S.: DDoS attacks at the application layer: challenges and research perspectives for safeguarding web applications. IEEE Commun. Surv. Tutorials **21**(1), 661–685 (2018)
4. Masdari, M., Jalali, M.: A survey and taxonomy of DoS attacks in cloud computing. Secur. Commun. Networks **9**(16), 3724–3751 (2016)
5. Dong, S., Abbas, K., Jain, R.: A survey on distributed denial of service (DDoS) attacks in SDN and cloud computing environments. IEEE Access **7**, 80813–80828 (2019)
6. Tripathi, N.: Delays have dangerous ends: slow HTTP/2 DoS attacks into the wild and their real-time detection using event sequence analysis. IEEE Trans. Dependable Secure Comput. (2023)
7. Khraisat, A., Gondal, I., Vamplew, P., Kamruzzaman, J.: Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecurity **2**(1), 1–22 (2019)
8. De Neira, A.B., Kantarci, B., Nogueira, M.: Distributed denial of service attack prediction: challenges, open issues and opportunities. Comput. Netw. **222**, 109553 (2023)
9. Agrafiotis, G., Kalafatidis, S., Giapantzis, K., Lalas, A., Votis, K.: Advancing cybersecurity with AI: a multimodal fusion approach for intrusion detection systems. In: 2024 IEEE International Mediterranean Conference on Communications and Networking (MeditCom), pp. 51–56. IEEE (2024)
10. Hasanov, I., Virtanen, S., Hakkala, A., Isoaho, J.: Application of large language models in cybersecurity: a systematic literature review. IEEE Access (2024)
11. Cilium-hubble. https://docs.cilium.io/en/stable/overview/intro/
12. Kalafatidis, S., Mamatas, L.: Microservices-adaptive software-defined load balancing for 5G and beyond ecosystems. IEEE Network **36**(6), 46–53 (2022)
13. Kalafatidis, S., Agrafiotis, G., Giapantzis, K., Lalas, A., Votis, K.: Experiments with digital security processes over SDN-based cloud-native 5g core networks. In: 2024 27th Conference on Innovation in Clouds, Internet and Networks (ICIN), pp. 97–99. IEEE (2024)

14. "Floodlight SDN Controller". https://github.com/floodlight/floodlight
15. "Containernet". https://containernet.github.io
16. "tshark". https://tshark.dev/
17. "Together AI platform". https://www.together.ai/

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.